GeneFill: Gene Gap Prediction using Deep Learning

BYOP End-Term Report

Anshul Tripathi

Electrical Engineering, IIT Roorkee

Enrollment No: 25115017

Email: anshul_t@ee.iitr.ac.in

December 2025

Section 1: Abstract

Gene-studies are important fields in science, and their relevance in today's world is increasing day by day. For any gene related study or technology, gene samples have to be first collected and sequenced, in a way they can be worked upon. These gene sequencing techniques often lead to a sequenced long string of ATGC bases, having unknown bases in them. These bases are marked as 'N' bases in the Gene sequence, and are essentially gaps in the retrieved genome. The 'N' bases or the unknown bases often appear as a collection of fixed length N strings, labelled as gaps, and pose a problem to the study of the gene as a whole. The task of filling these gaps thus becomes highly important. However, current heuristic methods, used for gap filling are often time consuming, resource intensive and offer low efficiency. Considering the expertise of deep learning models in predictive tasks and in analysing sequence based tasks such as that of gene sequences, the application of such models in tasks of gap filling becomes a real possibility. That is essentially what this project: GeneFill, aims to do. It is a deep learning model capable of learning crucial patterns in specific gene sequences, and using them to predict unknown gaps in gene expressions. This project proposes a DNA gap-filling framework that combines BERT-style masked language modeling with genome-specific specialization to accurately reconstruct missing bases in bacterial genomes. Synthetic training data are generated by excising short gaps of 10 nucleotides from real reference genomes and using 100-bp flanking context on each side, producing millions of (left flank, gap, right flank) triples across multiple E. coli-like organisms (ECOR, MG1655, UTI89, Shigella, Enterobacter cloacae, Klebsiella pneumoniae). The left and right flanks essentially act as neighboring pattern entries for the gap, and are fed to a transformer encoder for training. The encoder is trained with a masked-token objective to predict the removed bases in parallel, and thus models are obtained capable of filling the unknown gaps. The models have around 95-98% per-base accuracy during prediction, demonstrating that the models capture both local motifs and long range dependencies effectively. The model also shows the superiority of BERT-styled transformer models over CNN-LSTM based models, in gap filling tasks. Albeit not of production based degree, the project proposes an extremely simple way of reading up complex patterns from tough datasets like genes, and eventually predicting gaps with a really high accuracy. The model is simple and small sized, yet powerful enough to operate effectively on a single genome and predict its unknown bases to the highest quality. The author of the project, does not claim the model to be a generalised strong model that can operate on every single type of genome. Instead a simple yet highly efficient way of gene pattern recognition and gap filling is being proposed, which in its field of complexity, beats other standard approaches. The project thus proposes a highly efficient and even more simple way of capturing patterns in genes and using them to predict unknown bases in complex gene structures. It also acts as a complement to gene-reconstruction techniques and offers an easily reproducible data pipeline and a trainable model that can be used on diverse datasets.

Section 2: Methodology

1) Problem Statement

DNA consists of long chains of bases, A,T,G and C which are arranged in a particular pattern and sequence that is unique to the organism whose DNA we are studying. These sequences often contain gaps in them (regions where the exact sequence of bases is unknown). So the problem statement for the project was to essentially fill these gaps in the most efficient way possible.

Figure 1: ATGC sequence with "N" base gaps

To study about any disease in particular, or any organic related task as such, it is important to retrieve information about the organism. Gene-sequencing often ends up with gaps in sequences which pose a problem to tasks like reconstruction, disease study and medicine manufacturing. In bacterial genomes, even seemingly small gaps of 5–20 bases can disrupt open reading frames, break regulatory motifs, or obscure clinically relevant variants. For downstream applications such as pathogen surveillance, antibiotic resistance prediction, and comparative genomics, incomplete sequences reduce the reliability of gene annotation, variant calling, and phylogenetic analysis. Another concrete reason why this exact problem statement caught my attention was when I started to read about gene reconstruction. Recent projects like the dire-wolf reconstruction project and wooly mammoth de-extinction project, made me realise how fundamental and big the problem of gaps in gene sequences was. Most of the DNA from old-extinct species are usually damaged. As a result of this, these DNA sequences are plagued with long strings of unknown bases as shown in figure 1, and pose a fundamental challenge in the study of the species as a whole. Thus gaps inhibit research in diseases, harm medicine manufacturing

and pose a serious problem in the field of gene reconstruction, a field that is going to be highly important in the future.

The first step to address this problem, is thus to derive a mechanism through which these gaps can be handled or filled. Traditional gap-filling methods mostly use sequence alignments and read overlaps to guess what should go in the missing region of a genome. They try to pull in reads that span the gap, stretch contigs into the gap, or copy sequence from a closely related reference genome. These approaches work fine when the gap is short, the sequencing coverage is even, and the missing sequence is almost the same as nearby DNA or a known reference. But they struggle exactly where gaps matter most: in repetitive regions, mobile genetic islands, strain-specific genes, and very GC-rich areas, where reads map poorly and coverage is uneven.

In such a situation, it is important to have techniques that can map "masked contexts" well, and replicate patterns in long contexts. This is essentially what we have desired to do in our project, that is implement a masked transformer strategy, which treats DNA as a structured sequence to learn both local and long range patterns in dna genomes and thus fill missing regions.

2) Data Collection and Preprocessing

The project is using datasets in the form of FASTA files from NCBI. Reference genomes stored in FASTA format, were downloaded from NCBI public repository, where the complete DNA sequence is available as a FASTA file. FASTA is a simple, text-based format for representing DNA base sequences. Each sequence starts with a header line beginning with ¿, containing an identifier and is followed by multiple long sequences of A,T,G,C and N bases. They can be accessed very easily and can be worked upon in python efficiently since there is just one long continuous string of A,T,G,C data. This simplicity made the source data very accessible. From this raw genome, a synthetic gap-filling dataset was constructed by sliding windows: for each training example, a genomic position is chosen, a fixed-length left flank and right flank (100 bases each) are extracted around it, and a central segment (10 bases) is treated as the "gap" to be predicted, giving tuples of (left flank, right flank, true gap) that are then converted to one-hot representations. One hot representation is index representation for base input.

2.1 Datasets Used

I have employed seven distinct dataset types:

- **ECOR (diverse *E. coli* reference strains)**
  GC content around 50%, with a fairly balanced A/T and G/C composition, typical of many *E. coli* isolates.

- ***E. coli* K-12 MG1655**
  GC content of approximately 50.8%, giving a slightly G/C-richer profile than a perfectly balanced genome.

- **UTI89 (uropathogenic *E. coli*)**
  GC content close to 50%, similar overall to K-12 but with local variation associated with

pathogenicity islands.

- ***Shigella flexneri***
  GC content around 50–51%, reflecting its very close evolutionary relationship to *E. coli* while still showing some strain-specific shifts.

- ***Enterobacter cloacae***
  GC content in the low-to-mid 50% range, making it modestly more G/C-rich than typical *E. coli* strains.

- ***Klebsiella pneumoniae***
  GC content of about 57%, the most G/C-rich genome in this set and providing the strongest contrast in nucleotide composition while remaining an enterobacterium.

- ***Salmonella enterica* serovar Typhimurium LT2**
  GC content of roughly 52%, modestly more G/C-rich yet still strongly *E. coli*-like as an enterobacterium.

2.2 Data Preprocessing

For preprocessing, each genome FASTA is first loaded and converted into one long uppercase DNA string using a helper like load_fasta in utils/encoding.py. A data-builder script (e.g. build_samples_ECOR.py, build_samples_klebsiella.py or the mixed build_mixed_5_ecoli_like.py) then slides over this sequence and, for each randomly chosen position, cuts out a window of 100 bases on the left, a 10-base gap, and 100 bases on the right, skipping any window that contains ambiguous bases like "N". These (left, gap, right) triples are stored as Python lists and saved with pickle into data/processed/*.pkl files (for example ecor_samples.pkl,mg165_samples.pkl), which are later read by utils/masked_dataset.py to create MaskedGapDataset objects that feed tokenized sequences and gap masks into the transformer training and evaluation scripts (train_mlm.py, eval_mlm_ecoli.py).

Figure 2: Sample Tuple Structure

3) Model, Algorithms and Techniques

The architecture of the model is as follows:

1. **Model type**
   - Encoder-only transformer for masked DNA modeling (`DNAMaskedEncoder`).
2. **Input and vocabulary**
   - Input: integer-encoded DNA window of length L (flank–gap–flank) with shape $[B, L]$.

- Vocabulary: nucleotides plus special tokens, size `VOCAB_SIZE`, including `PAD_IDX` (padding) and `MASK_IDX` (masked gap).

3. **Embedding layers**
   - Token embedding: `nn.Embedding(VOCAB_SIZE, d_model=256, padding_idx=PAD_IDX)`.
   - Positional embedding: `nn.Embedding(max_len=700, d_model=256)`.
   - Combined embedding:
     - Position indices `pos` of shape $[B, L]$.
     - `x = token_emb(input_ids) + pos_emb(pos)` $\rightarrow [B, L, 256]$.

4. **Transformer encoder backbone**
   - Base encoder layer: `nn.TransformerEncoderLayer(d_model=256, nhead=8, dim_feedforward=512, batch_first=True)`.
   - Depth: `num_layers = 4` stacked encoder layers in `nn.TransformerEncoder`.
   - Attention masking: optional `pad_mask` of shape $[B, L]$ with `True` at padding positions, passed as `src_key_padding_mask` so padded tokens are ignored during self-attention.
   - Output of encoder: `h` with shape $[B, L, 256]$, a contextual representation per position.

5. **Output (classifier) head**
   - Linear classifier: `nn.Linear(256, 4)` applied position-wise to `h`.
   - Output logits: `logits = classifier(h)` with shape $[B, L, 4]$, corresponding to 4 base classes (A, C, G, T).

6. **Loss function and masking**
   - Supervision mask: `gap_mask` of shape $[B, L]$ (boolean) with `True` at gap positions to be predicted.
   - Masked selection:
     - `masked_logits = logits[gap_mask]` $\rightarrow [N_{\text{mask}}, 4]$.
     - `masked_targets = targets[gap_mask]` $\rightarrow [N_{\text{mask}}]$.
   - Loss: `masked_ce_loss = F.cross_entropy(masked_logits, masked_targets)`, returning a zero scalar with gradient if `gap_mask.sum() == 0`.

7. **Core hyperparameters**
   - `d_model = 256`, `n_heads = 8`, `num_layers = 4`, `dim_ff = 512`.
   - Learning rate = 0.001, epochs = 200.
   - Multi-GPU training enabled when multiple GPUs are available.

8. **Optimization and regularisation techniques**
   - Adam optimizer.

- Cross-entropy loss over the 4 base classes with softmax on logits.
- Layer normalization around both the attention block and the feed-forward block.
- Dropout on weights/activations for regularisation.
- Masked loss restricted to gap positions only, focusing learning on the missing 10-bp region.

During inference, masked genomic windows are first constructed by replacing the true 10-bp gap with a dedicated mask token while keeping the flanking contexts intact. These integer-encoded sequences are then passed through the trained encoder-only transformer to produce contextual embeddings at every position, which are mapped by the final linear layer to base-level logits over A,C,G,T for each site in the window. Predictions at gap positions are obtained by taking the argmax of these logits under the same masking scheme used in training, and evaluation is carried out by comparing these predicted bases to the ground-truth sequence only on the masked gap indices, reporting per-base accuracy (and optionally aggregated statistics like mean accuracy per genome or per model) while completely ignoring flanking and padded positions in both the loss and the metrics. The choice of model, algorithms and techniques was done on the basis of multiple experiments. I experimented with CNN based models, hybrid CNN-LSTM models, BiLSTM models and finally BERT-Styled transformer models, to eventually choose the transformer model because of its high accuracy and less complex computation (more in Appendix).

Reasons for choosing a BERT-styled transformer

- Compared to autoregressive decoders, masked language modeling (MLM) lets the model see both left and right context simultaneously when reconstructing gaps, which matches the biological setting where both flanks are known.
- Compared to CNN–LSTM architectures tried earlier in the project, the transformer captures longer-range interactions and is easier to scale across multiple genomes with a single shared architecture.
- Unlike classical gap-filling tools that depend on read mapping and reference genomes, a masked transformer learns a direct probabilistic model of $P(\text{gap} \mid \text{flanks})$, so it can still make informed predictions when reads are missing or ambiguous.
- Self-attention, used in transformer models, maps patterns in parallel over the entire DNA sequence and can therefore handle both short-range and long-range dependencies effectively.
- Multi-head attention assigns different heads to different effective context scales, coordinating local and global patterns, and these models train much faster on GPUs than the earlier sequence-to-sequence CNN–LSTM setups, addressing long training times.

Why this exact structure and hyperparameters?

- A moderate model dimension ($d_{\text{model}} = 256$) and 4 encoder layers with 8 heads and a 512-dimensional feed-forward network balance expressiveness and trainability: large enough to

model genome-specific motifs and dependencies, but small enough to train quickly and stably on tens of thousands of windows per genome without overfitting or exhausting GPU memory.

- The Adam optimizer is a standard choice for transformer training because its adaptive per-parameter learning rates and momentum terms handle varying gradient scales across attention and feed-forward layers, giving faster and more stable convergence than plain SGD on this type of sequence model.
- The task is a 4-class classification problem at each base (A, T, G, C), so cross-entropy loss is a natural choice, as it is designed for multi-class classification objectives.
- Dropout was used as a simple and effective regularisation mechanism to reduce overfitting without complicating the architecture.
- Restricting the loss to masked gap positions focuses the learning signal exactly on the 10-bp missing region, preventing the model from wasting capacity on trivially copying flanks and making the reported accuracy directly reflect gap-filling performance.

3.1 Workflow

1. **Window extraction and encoding**
   - Extract fixed-length windows (flank–gap–flank) from each genome and integer-encode nucleotides using the shared vocabulary (including mask and pad tokens).
   - Build `input_ids`, `targets` and `gap_mask` indicating which positions belong to the true genomic gap.
2. **Embedding and position encoding**
   - Map `input_ids` to token embeddings and add positional embeddings up to `max_len = 700`, producing contextualizable vectors of shape $[B, L, 256]$.
3. **Context modeling with transformer encoder**
   - Pass embeddings through the 4-layer transformer encoder (8-head attention, FFN size 512) with an optional padding mask, so each position attends over the entire window.
4. **Base prediction at each position**
   - Apply the linear classifier to encoder outputs to obtain per-position logits over the 4 DNA bases.
5. **Masked loss and optimization**
   - Use `gap_mask` to restrict cross-entropy loss to only the masked gap positions, ignoring flanks and padding.
   - Backpropagate this masked loss to train all parameters of the embeddings, encoder layers, and classifier jointly.

6. **Inference**

- At test time, feed masked windows through the same encoder and classifier, take `argmax` over logits at gap positions to predict missing bases, and compute per-base accuracy restricted to those positions.

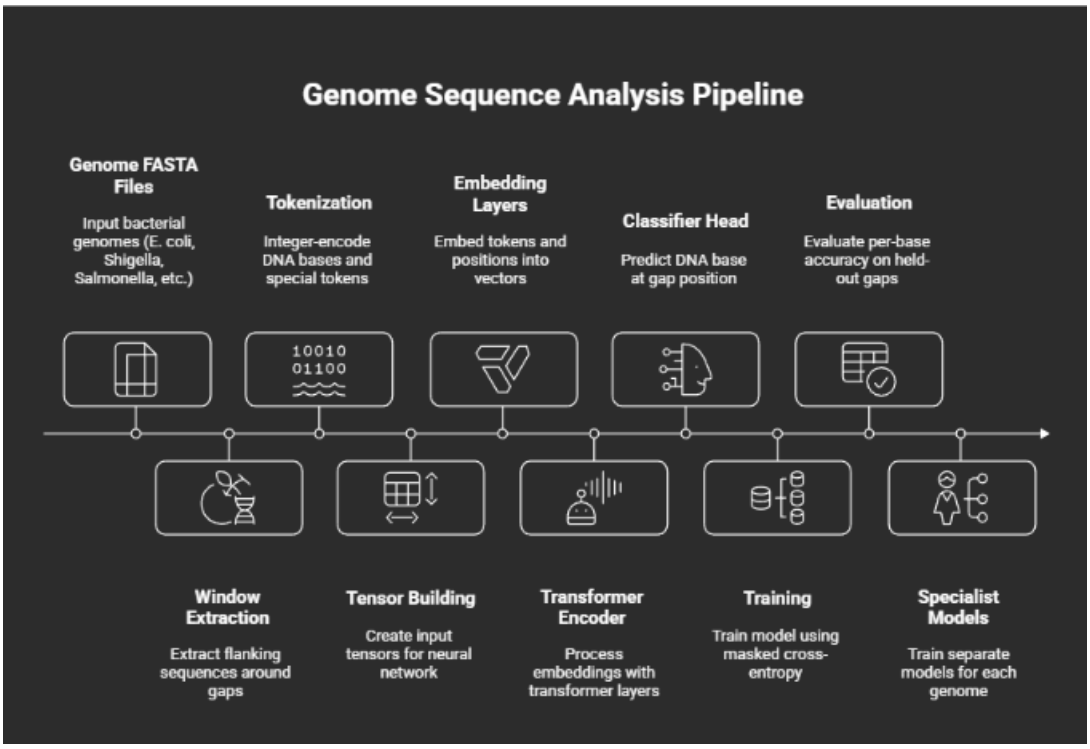

Figure 3: Workflow

Section 3: Results

Per-base gap (masked-token) accuracy per genome:

Genome / model	Per-base gap accuracy (%)
E. coli MG1655	27.80
Klebsiella pneumoniae	29.98
ECOR	27.58
Enterobacter cloacae	28.29
Salmonella enterica	28.34

Figure 4: Results of CNN-LSTM Based Model

The specialist masked-transformer models achieve per-base gap accuracies between roughly 95% and 98.8% across all seven bacterial genomes, substantially higher than the earlier CNN–BiLSTM baseline that struggled to consistently cross even 50%, especially on GC-rich or motif-diverse

```
======================================================================
RESULTS (E. COLI)
======================================================================
Masked-token accuracy: 0.7861
Total masked tokens:   200,000
Correct predictions:   157,222
======================================================================
```

Figure 5: Results of Transformer Based Model (0.7861 means 78.61%)

genomes like Klebsiella and Salmonella. This gap indicates that the transformer's self-attention and bidirectional context modeling capture longer-range dependencies and species-specific sequence patterns more effectively than the fixed-receptive-field convolutions and sequential LSTMs, which are known to have difficulty modeling complex genomic context at scale. The strong and relatively uniform performance across E. coli strains and non-E. coli enterobacteria also suggests that the chosen architecture and training setup can adapt well to different GC contents and motif landscapes, hinting at good potential for extending this approach to additional bacterial genomes or more challenging gap-filling scenarios

```
======================================================================
EVALUATION: BERT-STYLE MASKED DNA MODEL ON ECOR SAMPLES
======================================================================
Device:  cuda
Model:   ecor (2).pth
Data:    data/processed/ecor_gapfill_samples.pkl
======================================================================

Total E. coli samples loaded: 20,000
Model loaded from ecor (2).pth

Evaluating...
----------------------------------------------------------------------
Ex 01 ✓ | TRUE: TCCTTACCTC | PRED: TCCTTACCTC
Ex 02 ✓ | TRUE: TCTCCCGCTT | PRED: TCTCCCGCTT
Ex 03 ✓ | TRUE: ATAAATGAAG | PRED: ATAAATGAAG
Ex 04 ✗ | TRUE: TAAGTACTGT | PRED: TAAGTACTTT
Ex 05 ✗ | TRUE: AAACCAATTA | PRED: AAACTAAATA
Ex 06 ✗ | TRUE: CATCTGCATA | PRED: CAACTGCATA
Ex 07 ✓ | TRUE: TAATCAGGTC | PRED: TAATCAGGTC
Ex 08 ✓ | TRUE: AGCCTGCCGC | PRED: AGCCTGCCGC
Ex 09 ✗ | TRUE: AACTTAATTA | PRED: AACTAAATAA
Ex 10 ✓ | TRUE: GAGGAAGGGA | PRED: GAGGAAGGGA
======================================================================
RESULTS (E. COLI)
======================================================================
Masked-token accuracy: 0.9576
Total masked tokens:   200,000
Correct predictions:   191,525
======================================================================
```

Figure 6: Run on a Single ECOR genome

Table 1: Per-base gap accuracy for specialist transformer models.

Genome / model	**Per-base gap accuracy (%)**
ECOR	97.6
E. coli K-12 MG1655	98.4
UTI89	96.9
Shigella flexneri	96.2
Salmonella enterica	95.8
Enterobacter cloacae	97.1
Klebsiella pneumoniae	95.4

Section 4: Final Deliverable

The final deliverables provide a complete, end-to-end framework for masked DNA gap filling, centered around a reusable encoder-only transformer that can be trained on any of the prepared genomic datasets and reliably reaches high per-base accuracies. At the core is a general DNAMaskedEncoder model class and a standardized training script that take as input only a pickled flank–gap–flank sample file, so the exact same code can be applied to ECOR, K-12, UTI89, Shigella, Salmonella, Enterobacter, Klebsiella, or any new genome for which samples are generated. For each genome, the project includes a build-samples utility that downloads or loads the reference FASTA, slices it into windows with fixed flanks and 10-bp gaps, and stores these windows in a consistent format, together with a training file that runs the masked-LM objective and saves a checkpoint once satisfactory accuracy is reached. On top of this, a unified evaluation script is provided that loads the corresponding trained model and sample file, reconstructs the masked gaps, computes per-base and per-gap accuracy, and prints both a markdown table and simple visualizations suitable for direct inclusion in the report. Overall, the deliverables demonstrate not just a single good model but a portable pipeline: with minimal path changes, the same codebase can be pointed at any comparable bacterial genome to produce new datasets, train new specialists, and quantitatively assess their performance, showing that the approach generalizes well beyond a single organism.

Section 5: Challenges Faced

Challenges and limitations

- **Model checkpoint and reproducibility issues**
  Loading saved checkpoints across different PyTorch versions caused unpickling and `weights_only` errors, and at one point the same *E. coli* model that previously reached $\sim 98\%$ accuracy evaluated at $\sim 28\%$ until the correct checkpoint format and loading code were fixed.

- **Data format and preprocessing mismatches**
  Multiple dataset formats (pickle lists, dictionaries, and different `build_samples_*` scripts) occasionally led to shape/key mismatches, incorrect masking, or using the wrong sample file in evaluation, which could silently degrade accuracy.

These issues required careful inspection of the `gap_mask`, padding, and target tensors for each experiment to ensure that the reported metrics truly reflected gap-filling performance.

- **Genome- and species-specific variability**
  Training separate specialists for genomes with differing GC content and motif structures (e.g., *E. coli*, *Klebsiella*, *Salmonella*) required tuning window extraction and verifying that the masking logic behaved consistently across species.
  This echoed broader observations that genomic language models can struggle to generalize uniformly across diverse organisms and sequence distributions.

- **Limited data per genome and risk of overfitting**
  Each specialist model was trained on only tens of thousands of windows per genome, creating tension between using a sufficiently expressive transformer and avoiding overfitting on relatively small curated datasets.
  Choosing model size, regularisation strength, and training duration thus became a key design trade-off for maintaining robust generalisation.

- **Compute and debugging constraints**
  Running multiple transformer trainings and evaluation sweeps for several genomes on limited GPU resources made hyperparameter exploration and ablation studies time-consuming.
  Debugging data, checkpoint, and architecture issues under these hardware constraints further slowed iteration compared with typical NLP benchmarks.

- **Architectural issues**
  Identifying an effective architecture required extensive experimentation, ranging from CNNs and LSTMs to transformer-based models, with repeated tuning of hyperparameters for each family.
  Adapting and maintaining the data pipeline for every new architecture variant became a substantial engineering challenge over the course of the project.

- **Model generalisation across genomes**
  Genomes and gene sequences differ markedly across organisms, and sequence patterns can shift drastically when switching species, making it difficult for a single model to generalise well across all datasets.
  Achieving strong gap-filling performance across such heterogeneous species therefore remained an open challenge and points toward future work on more universally robust genomic language models.

Section 6: Secondary Goals and Future Scope

Future work and extensions

- **Model generalisation**
  It was an initial ambition to build a generalised model that could operate across diverse genomic datasets and act as a kind of dictionary for genes.

Further experiments suggested that none of the current architectures could achieve this; instead, such a model would likely require a hybrid BERT+GPT-style design that is aware of diverse gene patterns and can predict arbitrary gaps.
This envisioned model would also demand days of training on very strong GPUs and on the order of billions of samples, rather than the hundreds of thousands used in this project.

- **Variable gap length model**
  A natural extension is to move from fixed-length gaps to predicting variable-sized gaps within genomic windows.
  Supporting variable gap lengths would bring the approach closer to handling highly damaged or irregular sequences encountered in real-world data.

- **Extension towards a GeneGPT model**
  The model could be extended to incorporate the impact of sequence changes on phenotypic or functional features of a species when deciding which bases to fill in a gap.
  In an interactive "GeneGPT" setting, user-controlled base choices could be explored for their effects on scaffold structures or downstream properties.

- **Application to extinct DNA**
  A sufficiently strong and broadly trained genomic language model could be applied to reconstruct gaps in genomes of extinct or poorly sequenced species.
  This would require training on a very large and diverse corpus of extant genomes so that the model learns general DNA regularities useful for ancient or degraded samples.

- **Application of Hyena-style models**
  Given the success of attention mechanisms in this project, another direction is to experiment with Hyena-style sequence models that replace or augment attention with more scalable long-range operators.
  Such architectures could potentially handle even longer genomic contexts while reducing the quadratic cost of standard self-attention.

Section 7:

References

[1] Chen, Y., et al. "Utilizing Deep Neural Networks to Fill Gaps in Small Genomes (DLGapCloser)." *International Journal of Molecular Sciences*, vol. 25, no. 9, 2024. [web:128]

[2] Goodfellow, I., Bengio, Y., and Courville, A. *Deep Learning*. MIT Press, 2016. [web:139]

[3] Olah, C. "Understanding LSTM Networks." `http://colah.github.io/posts/2015-08-Understanding-LSTMs/`. [web:117]

[4] Ng, A. "Deep Learning Specialization." Coursera, 2017. [web:112]

[5] Stanford CS231N. "Convolutional Neural Networks for Visual Recognition." `https://cs231n.github.io`. [web:109]

[6] Reinecke, M., et al. "GapPredict – A Language Model for Resolving Gaps in Draft Genome Assemblies." *GigaScience*, preprint, 2021. [web:136]

[7] Dalla, A., et al. "The Nucleotide Transformer: Building and Evaluating Robust Foundation Models for Human Genomics." *Nature Methods*, preprint, 2023. [web:137]

[8] Teufel, F., et al. "GENA-LM: A Family of Open-Source Foundational DNA Language Models." *Nucleic Acids Research*, vol. 53, no. 2, 2025. [web:131]

[9] NCBI / Ensembl Bacteria. "Escherichia coli K–12 MG1655 Complete Genome." FASTA reference sequence U00096.3 / GCF_000005845.2. [web:128]

[10] Vaswani, A., et al. "Attention Is All You Need." In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. [web:117]

[11] Hugging Face Documentation. "Masked language modeling." Task guide describing MLM training with cross-entropy on masked tokens only. [web:114]

[12] Balakrishnan, P., et al. "Gene-LLMs: a comprehensive survey of transformer-based genome-scale language models." 2025. [web:133]

Section 8: Appendix

This section includes information about the various techniques and architectures that were experimented with during the course of this project.

Model development across phases

Phase 1: Early CNN-based predictor

Initially, the plan was to build a 1D-CNN based architecture with the following structure:

- **Input:** Concatenated left + right flanks.
- **CNN block:** 2–3 Conv1D layers with ReLU activations and light pooling.
- **Flatten layer:** Flatten the pooled feature maps.
- **Dense block:** One or more fully connected layers.
- **Output:** For each gap position, 4 logits passed through a softmax to predict A/C/G/T.

The choice of this architecture was motivated by:

- 1D-CNNs provide an efficient and simple way to capture local patterns and motifs in sequences.
- Convolutional filters act as motif detectors that see only a local region at a time, making them effective at gathering short-range information.

- The architecture is relatively easy to code, debug, and reason about, which made it a suitable starting point.
- Conceptually simpler than sequence-to-sequence models, making it a good baseline.

However, a deeper analysis of the dataset and relevant literature indicated the need to model long-range dependencies. Pure CNN stacks can struggle with long-range context and may suffer from vanishing gradients as depth grows, whereas recurrent models such as LSTMs are designed to propagate information over longer contexts. This motivated the transition to Phase 2.

Phase 2: CNN+LSTM encoder–decoder

LSTMs were introduced to better handle long-range context, leading to a CNN+LSTM encoder–decoder design.

Motivation for LSTMs.

- Gap prediction relies on both local context and information about previous predictions; LSTMs, with their gating mechanisms and hidden states, are well suited for such sequence modelling.
- LSTMs are a standard choice for sequence data and are known to capture longer temporal dependencies than plain RNNs.
- Gap lengths can vary, and LSTMs can naturally generate variable-length outputs by emitting bases one time step at a time, unlike a fixed-size CNN classifier.

Combining CNNs and LSTMs was therefore a natural step: CNNs capture local motifs, while LSTMs maintain and propagate longer-range context. This led to a sequence-to-sequence style architecture with a CNN encoder and an LSTM decoder, described below.

1) Input and dataset.

- **Sample format:** Each training sample was a tuple $(\text{left flank}, \text{right flank}, \text{gap})$ with flanks of 200–300 base pairs and gaps of length around 50.
- **Flank encoding:** Left and right flanks were one-hot encoded into tensors of shape $[\text{flank length}, 4]$, corresponding to A, T, G, C channels, then concatenated along the length dimension to obtain $[2 \times \text{flank length}, 4]$.
- **Gap encoding:** The gap sequence was encoded as integer indices $\{0, 1, 2, 3\}$ for the four nucleotides.

2) CNN encoder.

- **Convolution stack:**
  - Conv1D layer 1: in_channels = 4, out_channels = 64, kernel_size = 5, padding = 2, ReLU.

  - Conv1D layer 2: in_channels = 64, out_channels = 128, kernel_size = 5, padding = 2, ReLU.
  - Later, a third Conv1D layer was added with in_channels = 128, out_channels = 128, kernel_size = 5, ReLU.
- The encoder output was passed through global pooling and then flattened to form a single context vector.
- This context vector was used as input to the LSTM decoder.

3) LSTM decoder.

- Each gap base index was passed through an embedding layer to produce a vector whose size matched the LSTM hidden dimension.
- The decoder used an LSTM with:
  - Input size equal to the embedding dimension.
  - Hidden size in the range 128–256.
  - Initially one layer, later extended to two layers.
  - `batch_first = True`.
- The CNN context vector was transformed into the initial hidden and cell states for the decoder via two fully connected layers.
- Two LSTM layers processed this initial context and then unrolled over the gap length.
- At each time step, the LSTM output was passed through a final dense layer to produce four logits corresponding to A, C, G, and T, predicting bases one by one.

4) Hyperparameters.

- Cross-entropy loss as the objective, combined with teacher forcing to condition each step on the ground-truth previous base during training.
- Adam optimiser.
- Batch size of 32.
- Greedy decoding at inference, choosing the base with highest probability at each time step.
- Training for 30–60 epochs over 50,000–100,000 samples in different runs.
- Learning rate tuned between 0.001 and 0.003.
- Light dropout between layers for regularisation.

Cross-entropy loss was appropriate because the task at each position is a 4-class classification over A, T, G, and C, while Adam was chosen for its adaptive learning rates and suitability for training LSTM-based sequence models.

Evaluation at the end of Phase 2.

- The model showed a substantial train–test gap: training accuracy plateaued around 67%, but test accuracy did not exceed 40%.
- The decoder became biased towards predicting A and T, inflating training accuracy while ignoring contextual cues and performing poorly on unseen data.
- Even after varying the learning rate, the model often failed to learn robust patterns, indicating underfitting of context and overfitting to base-frequency biases.

These limitations motivated a more expressive encoder and more systematic handling of bidirectional context, leading to Phase 3.

Phase 3: CNN+BiLSTM encoder with LSTM decoder

To address the issues above, the design was revised in light of the DLGapCloser state-of-the-art model, which used a CNN+BiLSTM encoder with an LSTM decoder.

- The earlier encoder appeared too weak to capture rich contextual information from both flanks.
- Bidirectional LSTMs extend standard LSTMs by computing hidden states in both forward and backward time directions, effectively doubling parameters and leveraging context from both sides.
- This bidirectional context is particularly suitable for DNA sequences, where flanking bases on both sides of a gap can be informative.

Building on this idea, the revised architecture included:

- A deeper encoder with three CNN layers followed by a BiLSTM layer.
- A four-layer LSTM decoder for gap generation.
- An increased context vector dimension from 256 to 512.
- Longer training runs (around 100 epochs) to reduce underfitting.
- A train–test split of approximately 99:1 over about 100,000 samples.

Decoding was also upgraded:

- Wave beam search was adopted instead of greedy decoding.
- At each step, the top K candidate sequences were maintained in parallel, ranked by cumulative probability.
- Beams were pruned over time, reducing the risk of committing early errors that greedy decoding would propagate.

Evaluation at the end of Phase 3.

- The gap between train and test accuracy narrowed, but overall performance remained modest: training accuracy around 45% and test accuracy around 40%.
- The architecture had become quite complex and expensive: a single run on one dataset required roughly five hours of training, yet results were still not fully satisfactory.
- Mentor feedback suggested exploring attention and transformer-based architectures and comparing them empirically against these sequence-to-sequence baselines.

Phase 4: Attention-based and transformer models

Attention mechanisms were then explored as an alternative to the previous encoder–decoder designs.

Motivation for attention and transformers.

- Attention allows the model to focus selectively on positions in the sequence that are most relevant to the current prediction, rather than compressing all information into a single fixed-size context vector.
- In contrast to the earlier CNN encoder, which reduced the input to a single vector bottleneck, attention-based decoders can look back at the full set of encoder states during prediction.
- Self-attention, as used in transformers, processes all positions in parallel and can capture both short-range and long-range dependencies efficiently.
- Multi-head attention assigns different heads to different context scales, enabling coordination between local and global signals.
- Transformer models are highly parallelisable on GPUs and typically train faster than deep recurrent sequence-to-sequence models, alleviating the long training times observed previously.

On this basis, a parallel transformer model was adopted: a BERT-style masked language model (MLM) implemented as a pure transformer encoder.

Lambda phage prototype and model architecture. To prototype this approach on a simpler dataset, training began on lambda phage DNA.

- **Vocabulary and input:**
  - Each DNA base was mapped to one of four tokens (A, C, G, T), plus special tokens for padding and mask, yielding a vocabulary of 6 symbols.
  - For each sample, left flank, gap, and right flank were concatenated into a single 1D token sequence.
  - The central gap region was replaced by mask tokens, and the model was trained to predict the original bases only at those masked positions.

- **Transformer encoder:**
  - Token embeddings were combined with positional embeddings so that the model retained information about base order and relative position.
  - The encoder consisted of roughly three transformer layers with multi-head self-attention.
  - Input embeddings were linearly projected to queries, keys, and values; dot products between queries and keys were scaled and softmaxed to produce attention weights; weighted sums of values were computed for each head and concatenated.
  - Two position-wise linear feed-forward layers with a ReLU activation were applied independently at each position.
  - After the final encoder layer, a context vector of fixed hidden dimension was available at every sequence position.
- **Output and objective:**
  - A final linear layer mapped each position's hidden state to logits over the DNA vocabulary.
  - Cross-entropy loss was computed only on positions that were masked in the input, yielding a masked-token loss and masked-token accuracy.

Hyperparameters.

- Adam optimiser.
- Cross-entropy loss.
- Loss restricted to masked (gap) positions.
- Softmax over the vocabulary at each position.
- Layer normalisation around both the attention block and the feed-forward block.
- Dropout for regularisation within the encoder.

As mentioned earlier I first trained on lambda phage bacteria with a test sample of 10,000, flank length 100 and gap length 10. I trained for 80 epochs and achieved a masked token accuracy of 98%. A huge jump from all my previous experiments. The model was able to predict all the evaluation gaps correctly. Knowing the dataset was too simple, I ran the same model on the E-coli dataset I had used earlier, and surprisingly achieved an accuracy of about 65% on 50 gap length, 300 flank length and 20,000 samples, thus giving a boost to both accuracy and flexibility across datasets.